

WHAT THIS SESSION IS

Run on 19 September 2026, and it is the session where MCP stops being theory. Akshay Agarwal builds everything from an empty folder: a FastMCP server, a LangChain client that starts it, and then four separate demos kept in four separate folders so that each idea has its own clean code. Along the way three things surprise the room. The server starts more than once for a single run, a shopping cart forgets what you put in it, and switching to HTTP does not fix that. Each surprise has the same answer underneath, which is sessions. The session ends with progress and log notifications, which is the first time the notifications from session 8 show up as real output on a screen.

CONTENTS

1. The plan: four folders, four ideas
2. Setting up the project
3. A server in five lines
4. What the model actually sees about a tool
5. The three invokes
6. The client, line by line
7. Reading the response and calling the tool
8. Two tools in one message
9. The surprise: counting how often the server starts
10. Why it happens
11. State on the server: Context
12. The client side of a session
13. What a session actually buys you
14. Can one tool call another tool?
15. The same server over HTTP
16. HTTP does not give you sessions for free
17. Notifications: logs and progress
18. Catching them on the client
19. Two servers, one client
20. Errors seen live, and the fixes
21. Clarifications from the Q&A

1 The plan: four folders, four ideas

Last session ended with a half working client and a dependency problem. This one starts from an empty folder on purpose. Nothing is carried over. The instructor also changed the structure: instead of one project that keeps growing, each topic gets its own folder with its own `server.py` and `client.py`. One idea per folder, so nothing from an earlier demo can confuse a later one.

Folder	What it shows
<code>01_first_mcp_call</code>	A stdio server, a client, and one full tool call from question to answer
<code>02_session_mcp</code>	State that survives between tool calls, using <code>Context</code> and a session
<code>03_call_mcp_http</code>	The same thing over HTTP instead of stdio
<code>04-notification</code>	Log and progress notifications, from both a stdio and an HTTP server

2 Setting up the project

```
uv init                # pyproject.toml, main.py, .python-version
uv sync                # creates .venv

uv add fastmcp          # to build MCP servers
uv add langchain-core    # messages
uv add langchain-mcp-adapters # MCP tools as LangChain tools
uv add langchain-openai  # the model
uv add mcp               # the official protocol SDK
uv add python-dotenv     # reads .env
```

Then a `.env` file with the key. Only the client needs it. The servers have no model in them and need no key at all.

```
OPENAI_API_KEY=sk-...
```

THE VERSION PROBLEM, AND HOW IT WAS SOLVED THIS TIME

The first run of the client failed again, the same way it did in session 8, and again it was a version mismatch between packages rather than anything wrong in the code. This time the instructor did not debug it live. He pasted a known good dependency block into `pyproject.toml` and ran `uv sync`. That is the practical fix: when a set of versions is known to work together, pin it and move on.

The block that the shared code ends up with:

```
dependencies = [  
    "fastmcp<4",  
    "langchain-core>=1.6.3",  
    "langchain-mcp-adapters>=0.3.1",  
    "langchain-openai>=1.6.2",  
    "mcp<2",  
    "python-dotenv>=1.2.3",  
]
```

Note `fastmcp<4` and `mcp<2`. Those upper bounds are there on purpose. FastMCP 4 changes several things covered in this class. To see what you actually have installed, use `uv pip list`, or just read `pyproject.toml`.

3 A server in five lines

```
from fastmcp import FastMCP  
  
mcp = FastMCP('FirstMCPServer')  
  
if __name__ == "__main__":  
    mcp.run(transport='stdio', show_banner=False)
```

That is a complete MCP server. It has no tools yet, but it runs, speaks the protocol and can be connected to.

- `FastMCP('FirstMCPServer')` creates the server. The name is what the client sees during initialization.
- `transport='stdio'` picks the transport from session 8. The other option is `'http'`, which comes in section 15. There is no third option. As one student asked, JSON-RPC is not a transport. It is the message format that both transports carry.
- `show_banner=False` hides the large FastMCP logo that otherwise fills the terminal every time the server starts. Small thing, but in this class it matters, because counting how often the server starts becomes the whole point of section 9.

Now a tool:

```
@mcp.tool
def greet(name: str) -> str:
    '''This is Greet Tool, and based on the name, it going to greet the user'''
    return f'Hello {name}, Welcome to telusko! How are you doing today?'
```

Without the decorator this is an ordinary Python function. `@mcp.tool` is what registers it with this server and makes it visible to clients.

4 What the model actually sees about a tool

This was drawn out carefully in class, and it is worth being exact about. When the model decides whether to call a tool, it sees only what the server publishes in `tools/list`:

THE SERVER SENDS

```
name:      greet
docstring:  "This is Greet Tool..."
parameters: name: str
return type: str
```

THE MODEL DECIDES WITH

```
the name
the description
the input schema
the output schema
```

THE SERVER DOES NOT SEND

```
the body of the function
the API it calls inside
the database it touches
your secrets
```

So the docstring is not a comment for your teammates. It is the sales pitch the model reads. A vague docstring means the tool does not get called, or gets called at the wrong moment. The type hints matter for the same reason: they become the schema that tells the model what to send.

WHY THIS MAKES MCP MORE ROBUST THAN CALLING AN API DIRECTLY

A question near the end of class asked whether MCP is immune to source code changes. It is not immune, but it is more forgiving, and this is why. If the team that owns the server renames `add` to `insert` and updates the description to match, your code does not break, because you never wrote the function name anywhere. Your model reads the new list at runtime and picks the right tool from the new description. With a REST client you would have hardcoded the endpoint and the payload, and you would be fixing it by hand.

5 The three invokes

Before writing the client, the instructor drew the picture that the rest of the session hangs on. Everything in a tool call is three calls to `invoke`, and it helps to know which is which.

```
user question
|
| INVOKE 1 (to the model)
v
+-----+ "I cannot answer this myself, but the server has a tool
| LLM | called greet that takes a name. Call it with name=Akshay."
+-----+
|
| INVOKE 2 (to the tool, through the MCP client)
v
+-----+ MCP client ---> MCP server ---> greet("Akshay")
| MCP client | <----- "Hello Akshay, Welcome..."
+-----+
|
| INVOKE 3 (to the model again, optional)
v
+-----+ turns the raw tool output into a proper answer,
| LLM | with the earlier messages as context
+-----+
|
v
final answer to the user
```

- **Invoke 1** does not run anything. The model only reports which tool it wants and with which arguments.
- **Invoke 2** is the actual work. Your code, not the model, calls the tool.

- **Invoke 3** is optional. The instructor was clear about it: if you just want the tool's output, print it and stop. You need the third call when you are building a chat and want the model to phrase the answer with the whole conversation in view. Several demos later in the session skip it entirely.

6 The client, line by line

01_first_mcp_call/client.py

```
from dotenv import load_dotenv
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain_openai import ChatOpenAI
from langchain_core.messages import AIMessage, HumanMessage, ToolMessage

import sys
from pathlib import Path
import asyncio

load_dotenv()

SERVER_PATH = Path(__file__).parent / "server.py"

async def main():
    client = MultiServerMCPClient({
        'greeting': {
            'transport': 'stdio',
            'command': sys.executable,
            "args": [str(SERVER_PATH)]
        }
    })

    tools = await client.get_tools()          # to get the tools list

    llm = ChatOpenAI(model="gpt-4o-mini")
    llm_with_tools = llm.bind_tools(tools=tools)

    messages = [HumanMessage(content="Print Akshay in unique way default times")]

    response = await llm_with_tools.ainvoke(messages)
    messages.append(AIMessage(content=response.content,
                               tool_calls=response.tool_calls))
```

- `'greeting'` is a label you pick for this connection. You can list several servers in the same dictionary, which is exactly what section 19 does.
- `sys.executable` is the Python running the client, so the server starts inside the same virtual environment. `Path(__file__).parent` makes the path work from any directory.
- `await client.get_tools()` starts the server, does the handshake and asks for the tool list. All of the lifecycle from session 8 happens inside this one line. A student asked which line does the handshake, and that is the honest answer: none of them visibly, all of it in the background.
- `bind_tools` is the same function from the tool calling classes. MCP changes where the tools come from, not how you give them to a model.
- `ainvoke` is the async version of `invoke`. Since `get_tools` and the MCP tools are async, the whole function is async and is started with `asyncio.run(main())`. If async and await are still hazy, the course has a video on coroutines in the Python gaps module, which the instructor pointed to during class.

A SMALL SIMPLIFICATION

The class rebuilds an `AIMessage` from the response with

`AIMessage(content=response.content, tool_calls=response.tool_calls)`. You can append `response` itself, since it already is an `AIMessage`. Both work. The instructor also noted mid-class that `messages` would be better named `history`, because that is what it is.

7 Reading the response and calling the tool

When the first response comes back, `content` is empty and `tool_calls` has something in it. That is the model saying it does not want to answer in words, it wants a tool run.

```
tools_by_name = {tool.name: tool for tool in tools}

for tool_call in response.tool_calls:
    tool = tools_by_name[tool_call["name"]]
    result = await tool.ainvoke(tool_call['args'])    # to call the tool
    tool_output = result[0]['text']
    print(tool_output)

    messages.append(ToolMessage(content=tool_output,
                                tool_call_id=tool_call['id']))

final_response = await llm_with_tools.ainvoke(messages)
print(final_response.content)
```

The dictionary comprehension caused the most confusion in the room, so it was printed out and explained twice. It is a lookup table: the key is the tool's name, the value is the whole tool object.

```
tools    (a list)                tools_by_name  (a dictionary)

[ StructuredTool(greet...),      { 'greet' : StructuredTool(greet...),
  StructuredTool(add...) ]      'add'      : StructuredTool(add...) }

the model says:
  tool_calls = [ {'name': 'greet', 'args': {'name': 'Akshay'}, 'id': '...' } ]
                  |
                  v
tools_by_name['greet'] ---> the tool object you can actually call
```

- `tool_call['args']` comes from the model. You do not build the arguments yourself.
- `result[0]['text']` is needed because an MCP tool returns a list of content blocks, as covered in session 8. The first block holds the text.
- `ToolMessage(..., tool_call_id=tool_call['id'])` ties the result back to the call the model asked for. Without that id the model cannot match them.
- The final `ainvoke` sends the whole history, which is now three messages: the human question, the model's tool call, and the tool result.

8 Two tools in one message

To see whether one message can trigger two tools, two more were added to the server:

```
@mcp.tool
def unique_way_print(name: str) -> str:
    '''Use this tool to print name in unique way'''
    return 'Haha' + name

@mcp.tool
def default_number_of_printing() -> int:
    '''Use this tool to get the default times of printing'''
    return 5
```

With the message *"Print Akshay in unique way default times"*, the model called both. The loop in the client already handles that, because it iterates over `response.tool_calls` instead of taking only the first one.

WHAT THE FINAL ANSWER GOT WRONG

The tools returned `HahaAkshay` and `5`, and the model then answered that the name should be printed 5 times, without actually printing it 5 times. The instructor called this out rather than hiding it: the tools did their job, the model filled the gap between them with its own idea. The lesson is that combining tool outputs is not automatic. Either say it precisely in the prompt, or write a tool that does the whole job in one step.

9 The surprise: counting how often the server starts

With `show_banner=False` the server still prints one short line each time it starts. That line is what exposes the behaviour.

What the run does	How often the server starts
Load tools, model calls no tool	1
Load tools, model calls one tool	2
Load tools, model calls two tools	3

```
client.py                                server process

await client.get_tools()  -----> [start] handshake, tools/list [stop]

await tool.ainvoke(...)  -----> [start] handshake, tools/call [stop]

await tool.ainvoke(...)  -----> [start] handshake, tools/call [stop]

one run of one file, three separate server processes
```

10 Why it happens

By default the MCP client opens a fresh session for every single thing it does, and closes it straight after. With `stdio`, a new session means a new server process. The full lifecycle from session 8 runs each time: initialize, capabilities, initialized, work, shutdown.

The word for this is **stateless**. Nothing the server remembers survives, because the connection it belonged to is gone. A student asked what the point of the handshake is if this keeps happening, and the answer is that the handshake is exactly what has to be repeated to make a new connection valid.

IS THAT WASTEFUL?

Someone in class asked this about stdio, and it is a fair question. Starting a Python process per call is not free. For a local tool it is usually fine, and it keeps every call isolated. When it is not fine, the fix is a session, which is the next section. Over HTTP there is no process to start, but there is still a fresh handshake per call.

11 State on the server: Context

The example is a shopping cart, because it is the clearest case of something that must be remembered between calls. The new import is `Context`.

02_session_mcp/server.py

```
from fastmcp import FastMCP, Context

mcp = FastMCP('FirstMCPServer')
DEFAULT_PRODUCT_QUANTITY = 100

@mcp.tool
async def add_to_cart(item: str, context: Context):
    """Add an item to the shopping cart of the current session."""
    cart = await context.get_state("cart")

    product_quantity = await context.get_state("product_quantity")
    if product_quantity is None:
        product_quantity = DEFAULT_PRODUCT_QUANTITY

    await context.set_state('product_quantity', product_quantity - 1)

    if cart is None:
        cart = []
    cart.append(item)
    await context.set_state('cart', cart)
    return f"Added {item}. Cart now has {len(cart)} item(s)."
```

```
@mcp.tool
async def show_cart(context: Context):
    """Show everything in the shopping cart of the current session."""
    cart = await context.get_state("cart") or []
    return f"Cart: {cart}"
```

```
@mcp.tool
async def get_session_id(context: Context) -> str:
    """Return the ID of the current MCP session."""
    # Every session has a unique ID. Same session -> same ID.
    return context.session_id
```

- Add a parameter typed as `Context` and FastMCP fills it in for you. It is not something the client sends, and it does not appear in the tool's schema. The model never sees it.
- `await context.get_state(key)` and `await context.set_state(key, value)` read and write values. There is also `delete_state`.
- `get_state` returns `None` the first time, which is why both tools handle that case.

- `context.session_id` is the id of the current session. Same session, same id. It is the easiest way to prove to yourself whether two calls shared a session.
- The state is a plain key and value store. `cart` and `product_quantity` are two independent keys, which answers the question asked in class about whether you can keep more than one thing.

12 The client side of a session

The server side alone changes nothing. If every call still gets its own session, the state has nowhere to live. The client has to hold one session open and make all the calls inside it.

02_session_mcp/client.py

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain_mcp_adapters.tools import load_mcp_tools

client = MultiServerMCPClient({
    'cart_server': {
        'transport': 'stdio',
        'command': sys.executable,
        "args": [str(SERVER_PATH)]
    }
})

async with client.session("cart_server") as session:
    tools = await load_mcp_tools(session)
    tools_by_name = {}
    for tool in tools:
        tools_by_name[tool.name] = tool

    result = await tools_by_name["add_to_cart"].ainvoke({"item": "apple"})
    print(result[0]["text"])
    result = await tools_by_name["add_to_cart"].ainvoke({"item": "banana"})
    print(result[0]["text"])
    result = await tools_by_name["add_to_cart"].ainvoke({"item": "grapes"})
    print(result[0]["text"])

    result = await tools_by_name["show_cart"].ainvoke({})
    print(result[0]["text"])

    result = await tools_by_name["get_session_id"].ainvoke({})
    print(result[0]["text"])
```

Two changes carry the whole idea:

1. `async with client.session("cart_server") as session:` opens one session and keeps it open for the whole block. The name in the brackets must match the key in the dictionary above.
2. `load_mcp_tools(session)` instead of `client.get_tools()`. This is the part that is easy to miss. `get_tools()` would open its own session and the tools would not belong to yours.

Notice there is no model in this client at all. As the instructor pointed out, you are allowed to call MCP tools directly from your own code. Going through an LLM is a choice, not a requirement.

13 What a session actually buys you

Running the file gives one server start and a cart that remembers:

```
Added apple. Cart now has 1 item(s).
Added banana. Cart now has 2 item(s).
Added grapes. Cart now has 3 item(s).
Cart: ['apple', 'banana', 'grapes']
684e5d64-27c1-4296-9a2a-d68929f093dd
product_quantity: 97
```

Without the session block, the same calls print "Cart now has 1 item(s)" every time and `show_cart` comes back empty. The cart is created and thrown away three times over.

WITHOUT A SESSION

```
call 1 -> [session A] cart=[apple]
          session A closed
call 2 -> [session B] cart=[banana]
          session B closed
show    -> [session C] cart=[]
```

WITH A SESSION

```
+-- session S opens -----+
| call 1 -> cart=[apple]      |
| call 2 -> cart=[apple,banana]|
| call 3 -> cart=[.. ,grapes] |
| show    -> all three items  |
+-- session S closes -----+
```

`product_quantity` shows the same thing from the other side. It starts at 100 and each `add_to_cart` takes one away, so after three items it is 97. Run it without a session and it is 99 every time, because every call starts from 100 again.

WORTH KNOWING BEFORE YOU READ THE FASTMCP DOCS

This behaviour is what FastMCP 3 does, which is the version pinned in the class project (`fastmcp<4`). We ran the class code to confirm it, and the output above is from that run. In FastMCP 4 the picture changed: `ctx.set_state` is documented as living only for the current request, and anything that has to survive between tool calls moves to a separate session state API. So if you open the docs and read that state is request scoped, you are reading about the newer version, not about the code in this class.

14 Can one tool call another tool?

A student asked this during the cart demo, and it was tried live. A tool is a normal Python function with a decorator on it, so nothing stops you:

```

@mcp.tool
async def show_cart(context: Context):
    """Show everything in the shopping cart of the current session."""
    cart = await context.get_state("cart") or []
    add(2, 2)          # calling another tool as a plain function
    return f"Cart: {cart}"

@mcp.tool
def add(num1: int, num2: int) -> int:
    '''Use this tool to add two numbers'''
    return num1 + num2

```

That works. The first attempt did not, and the error is the interesting part. Calling `add_to_cart("mango")` from inside another tool failed with a missing argument for `context`. FastMCP fills in `Context` only when a tool is called *through the protocol*. Call it as a plain function and you have to supply everything yourself.

TWO DIFFERENT MEANINGS OF "TOOL CALLS TOOL"

What happened here is one function calling another inside the same file, which is just Python. The more interesting version is a tool on one server calling a tool on a *different* server, which needs an MCP client inside the server. The instructor said the idea is perfectly allowed and promised to show it. That is the first demo of session 10.

15 The same server over HTTP

Folder 03 is the same code with the transport swapped. On the server, one line changes:

```

if __name__ == "__main__":
    mcp.run(transport='http', host='127.0.0.1', port=8000)

```

The server now prints the address it is listening on, and the MCP endpoint is the URL plus `/mcp`:

```

http://127.0.0.1:8000/mcp

```

On the client, the command and args are replaced by a URL:

```

client = MultiServerMCPClient({
    'greeting': {
        'transport': 'streamable_http',
        'url': 'http://127.0.0.1:8000/mcp'
    }
})

```

Everything after that is unchanged. Same `get_tools`, same `bind_tools`, same loop. That is the payoff of a standard protocol: the transport is a detail in a dictionary.

THE MISTAKE EVERYONE MAKES ONCE

The first run failed, and the reason is the important difference between the two transports. With `stdio` the client starts the server for you. With `HTTP` it does not. The server is an independent process that has to be running already, in its own terminal:

```

# terminal 1
uv run 03_call_mcp_http/server.py

# terminal 2
uv run 03_call_mcp_http/client.py

```

	stdio	HTTP
Who starts the server	The client	You do, separately
Config keys	<code>command</code> , <code>args</code>	<code>url</code>
Transport value	<code>'stdio'</code>	<code>'streamable_http'</code>
Port	None	Yes, for example 8000
Speed on one machine	Faster, no network	Slower, goes through a port
Reachable from elsewhere	No	Yes

On the host value: `127.0.0.1` keeps the server on your own machine, which is the safe default. `0.0.0.0` makes it listen on every network interface so other machines can reach it. Use that only when you actually mean to expose it.

16 HTTP does not give you sessions for free

A very good question came from the room: now that we start the server by hand and it stays running, does the session take care of itself? It feels like it should. It does not.

The cart tools were copied into the HTTP server and called from the client without a session block. The result: every call reported one item, and `show_cart` came back empty. We reproduced this to be sure. Same server, same tools, only the client changed:

```
--- no explicit session (new session per call) ---  
Added apple. Cart now has 1 item(s).  
Added banana. Cart now has 1 item(s).  
show_cart: Cart: []  
  
--- inside one client.session() block ---  
Added apple. Cart now has 1 item(s).  
Added banana. Cart now has 2 item(s).  
show_cart: Cart: ['apple', 'banana']
```

THE LINE TO REMEMBER

A server process staying alive is not the same as a session staying open. The process is just a program that is running. The session is the agreed connection between one client and that server, and it is the thing state hangs on. Over HTTP the process outlives your calls, but each call still opens and closes its own session unless you hold one open.

17 Notifications: logs and progress

Session 8 described notifications as JSON-RPC messages with no id, sent without anyone asking. Folder 04 makes them visible. Picture a tool that takes a while, because it is calling an API, querying a database or running a model. The client should not sit in silence.

04-notification/server.py

```
import asyncio
from fastmcp import Context, FastMCP

mcp = FastMCP("Notification Server")

@mcp.tool
async def do_work(ctx: Context) -> str:
    """Do a slow job in 3 steps and report progress along the way."""

    for step in range(1, 4):
        await asyncio.sleep(1)

        # Notification 1: a log message.
        await ctx.info(f"Finished step {step}")

        # Notification 2: a progress update (progress out of total),
        # with a short text message that goes along with it.
        await ctx.report_progress(progress=step, total=3,
                                   message=f"Step {step} is done")

    # The normal answer. It arrives LAST, after all the notifications.
    return "Job done!"

if __name__ == "__main__":
    mcp.run(transport="stdio", show_banner=False)
```

- `await ctx.info(...)` sends a log message. There are also `ctx.debug`, `ctx.warning` and `ctx.error` for other levels.
- `await ctx.report_progress(progress=, total=, message=)` sends a progress update. Three out of three, with a line of text.
- The `return` is the ordinary result, and it arrives last. The notifications go out while the tool is still running.
- `asyncio.sleep(1)` is only standing in for real work.

The same file exists as `http_server.py` with the last line changed to `mcp.run(transport="http", host="127.0.0.1", port=8001)`. The tool body is identical, which is the point being made: notifications are a protocol feature, not a transport trick.

18 Catching them on the client

Notifications arrive outside the normal request and reply flow, so you cannot pick them up from the return value. You register handlers instead, through `Callbacks`.

```
from langchain_mcp_adapters.callbacks import Callbacks

async def on_log(params, context):
    print(f"LOG      : [{context.server_name} / {context.tool_name}]",
          params.data["msg"])

async def on_progress(progress, total, message, context):
    print(f"PROGRESS: [{context.server_name} / {context.tool_name}] "
          f"{progress:.0f} / {total:.0f} - {message}")

client = MultiServerMCPClient(
    { ... servers ... },
    callbacks=Callbacks(on_logging_message=on_log, on_progress=on_progress),
)
```

- `on_logging_message` receives everything sent with `ctx.info` and the other log levels. The text is in `params.data["msg"]`.
- `on_progress` receives `progress`, `total` and `message`, matching what the server sent.
- Both get a `context` that says **which server** and **which tool** the notification came from. With several servers connected that is what keeps the output readable.
- The function names are yours. Printing is the simplest thing to do with them. In a real app you would write them to a log file, push them to a UI, or update a progress bar.

19 Two servers, one client

The class had asked several times why the class is called `MultiServerMCPClient`. This demo answers it by connecting to both servers at once, one over stdio and one over HTTP.

```

client = MultiServerMCPClient(
    {
        "stdio_server": {
            "transport": "stdio",
            "command": sys.executable,
            "args": [str(SERVER_PATH)],
            "env": {"FASTMCP_LOG_LEVEL": "WARNING"}, # hides the server's own logs
        },
        "http_server": {
            "transport": "streamable_http",
            "url": SERVER_URL,
        },
    },
    callbacks=Callbacks(on_logging_message=on_log, on_progress=on_progress),
)

for server_name in ["stdio_server", "http_server"]:
    print(f"\n--- Calling do_work on {server_name} ---")

    tools = await client.get_tools(server_name=server_name)
    tools_by_name = {tool.name: tool for tool in tools}
    tool = tools_by_name["do_work"]

    result = await tool.ainvoke({})
    print("RESULT  :", result[0]["text"])

```

Start the HTTP server first (`uv run 04-notification/http_server.py`), then run the client. The output, which we reproduced with the class code:

```

--- Calling do_work on stdio_server ---
LOG      : [stdio_server / do_work] Finished step 1
PROGRESS: [stdio_server / do_work] 1 / 3 - Step 1 is done
LOG      : [stdio_server / do_work] Finished step 2
PROGRESS: [stdio_server / do_work] 2 / 3 - Step 2 is done
LOG      : [stdio_server / do_work] Finished step 3
PROGRESS: [stdio_server / do_work] 3 / 3 - Step 3 is done
RESULT   : Job done!

--- Calling do_work on http_server ---
LOG      : [http_server / do_work] Finished step 1
PROGRESS: [http_server / do_work] 1 / 3 - Step 1 is done
...
RESULT   : Job done!

```

CLIENT

SERVER (tool is running)

```

| tools/call do_work id=7 |
|----->|
| | step 1 done
| notifications/message (no id) |
|<-----| on_log fires
| notifications/progress (no id) |
|<-----| on_progress fires
| | ... steps 2 and 3 ...
| response id=7 "Job done!" |
|<-----| the await finally returns

```

`"env": {"FASTMCP_LOG_LEVEL": "WARNING"}` is a practical detail worth keeping. It sets an environment variable for the server process the client starts, so FastMCP's own startup chatter stays out of your output and you only see your own notifications.

20 Errors seen live, and the fixes

What appeared	Cause	Fix
Import error on the first client run	Mismatched package versions, as in session 8	Paste a known good dependency block into <code>pyproject.toml</code> , then <code>uv sync</code>
Warning about a coroutine never awaited, no output	<code>ainvoke</code> called without <code>await</code>	Add <code>await</code>
<code>'list' object has no attribute 'text'</code>	The tool result is a list of content blocks	<code>result[0]['text']</code>
<code>Couldn't find a server with name cart_server</code>	The name in <code>client.session("cart_server")</code> did not match the key in the config	Use the same name in both places
<code>ClientSession object has no attribute get_tools</code>	A session is not the client. It has no <code>get_tools</code>	<code>await load_mcp_tools(session)</code>
HTTP client could not connect	The HTTP server was not running	Start the server in its own terminal first
<code>missing 1 required argument: context</code>	A tool that takes <code>Context</code> was called as a plain function	Call it through the protocol, or split the logic into a plain helper

21 Clarifications from the Q&A

- **Can JSON-RPC be the transport?** No. JSON-RPC is the message format. stdio and HTTP are the transports that carry it.
- **Which line does the handshake?** None that you write. It happens inside `get_tools`, inside a session, and inside every tool call that opens its own session.
- **Does the LLM talk to the MCP server?** No. The model only names a tool. Your MCP client is what talks to the server.
- **Is the third invoke necessary?** No. Skip it when you just want the tool output. Keep it for chat, where the answer should be phrased with the history in view.
- **What if the message history gets too long?** That is what the summarisation middleware from session 6 is for, among other approaches.

- **Which is faster, stdio or HTTP?** stdio, when everything is on one machine. HTTP is the choice when the server has to be reachable from outside.
- **Can we use a local `@tool` and a third party MCP server together?** Yes. Load both sets of tools and bind them to the same model.
- **Are these global states?** Yes, the state is per session, not per user. There is no per user state here, and no authentication yet. That comes in session 10.
- **Can an agent run the tools instead of us?** Yes, `create_agent` runs this loop for you. The loop is written by hand here so that every step is visible.
- **Can the server be in Docker?** Yes. For stdio the command and args become the docker command. For HTTP it is just a different URL.
- **How do we know which library versions are right?** Check with `uv pip list`, keep the working versions pinned in `pyproject.toml`, and reuse that block across projects.

WHAT IS LEFT FOR TOMORROW

Announced at the end of class: error handling, authentication and authorization with roles, and using an MCP server that somebody else wrote. After MCP the course moves to RAG, then to end to end projects.

22 Key takeaways and official links

- **A FastMCP server is five lines.** Create it, decorate functions with `@mcp.tool`, run it with a transport.
- **The model sees name, description and schema.** Never the body. Write the docstring for the model.
- **Three invokes:** ask the model, call the tool, optionally ask the model again.
- **Your code calls the tool,** not the model. The model only says which one and with what.
- **Results come back as content blocks,** so read `result[0]['text']`, and tie them back with `tool_call_id`.
- **By default every call is its own session,** which is why the server starts once per call over stdio.
- **State lives in a session.** `Context` with `get_state` and `set_state` on the server, `client.session(...)` plus `load_mcp_tools` on the client.
- **A running server is not an open session.** Over HTTP you still need the session block.
- **Switching transport is a config change,** but with HTTP you start the server yourself.
- **Slow tools should talk:** `ctx.info` for logs, `ctx.report_progress` for progress, and `Callbacks` on the client to catch them.
- **One client can hold many servers,** mixing stdio and HTTP freely.

Official links

What	Link
FastMCP documentation	gofastmcp.com
FastMCP Context (logging, progress, state)	gofastmcp.com/servers/context
Running a server (stdio and HTTP)	gofastmcp.com/deployment/running-server
langchain-mcp-adapters	github.com/langchain-ai/langchain-mcp-adapters
LangChain MCP guide	docs.langchain.com/oss/python/langchain/mcp
MCP specification	modelcontextprotocol.io/specification/latest

BEFORE NEXT CLASS

Run all four folders yourself. Then try three things. Count the server start lines with and without a session, so the stateless behaviour is something you have seen rather than read. Add a second key to the cart state, such as a discount code, and show it from a new tool. Finally, add `ctx.info` calls to any tool you already wrote and watch them arrive before the result.